

# My AI Engine

Motor personal de pensamiento para uso en Inteligencia Artificial — Manual de uso v2.0

autor: riv4wi — Mayo 2026

## 1. Qué es esto

Este motor es un framework minimalista de archivos de texto que actúan como **sistema operativo** del asistente de IA. En lugar de explicar cada vez tus reglas, estilo y arquitectura, le entregás los archivos al inicio de la sesión y la IA arranca configurada.

La filosofía central es la **separación entre el motor y los proyectos**: este repositorio contiene únicamente los 3 archivos núcleo genéricos y reutilizables. Los archivos específicos de cada proyecto (contextos, arquitecturas, decisiones de BD) viven dentro del repositorio de código de cada proyecto individual, no aquí.

### Beneficios

- **Cero repetición**: tus reglas viven en archivos, no en la memoria de un chat.
- **Portabilidad**: el mismo motor funciona en Claude, ChatGPT, Gemini, DeepSeek y modelos locales.
- **Token-eficiente**: el SoT bajo trigger evita planes largos en tareas triviales.
- **Trazable**: cada decisión del agente puede rastrearse a una regla declarada.
- **Escalable**: cada proyecto instancia sus propias reglas sin contaminar el motor central.

## 2. Los 3 archivos núcleo

El motor consta de exactamente 3 componentes. Son genéricos y reutilizables en cualquier proyecto sin modificación.

| Archivo                       | Tipo                  | Descripción                                                                                                                                                                                                       |
|-------------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| master_prompt.md              | Universal             | Núcleo inmutable. Instrucciones base, filosofía, formato de salida y reglas de seguridad. Se carga en <b>todas</b> las sesiones, siempre.                                                                         |
| preset_stack.md               | Plantilla de stack    | Plantilla base para crear módulos de tecnología. Define mejores prácticas, reglas de sintaxis y arquitectura para un lenguaje o framework. Se instancia por proyecto (ej: preset_laravel.md, preset_python.md).   |
| contexto_proyecto.template.md | Plantilla de proyecto | Plantilla de inicio de proyecto. Se instancia una vez por proyecto con toda la información vital: stack, rutas, arquitectura, antipatrones, decisiones tomadas. Vive en el repo del proyecto, no en my-ai-engine. |

Regla mental: **master** → **preset** → **contexto**. Los tres se cargan juntos para cualquier sesión técnica. El preset y el contexto vienen del repo del proyecto. Solo el master viene de my-ai-engine.

## Ejemplo de flujo de trabajo en un proyecto real

Supongamos que trabajás en un proyecto **apibml** que usa **Laravel** y **Oracle**. Los archivos instanciados viven en el repo del proyecto, no en `my-ai-engine/`:

### Paso 1 — Preparación (una sola vez por proyecto)

- Copiás `preset_stack.md` a tu proyecto como `preset_laravel.md` y lo completás con reglas de PHP/Laravel.
- Copiás `contexto_proyecto.template.md` a tu proyecto como `contexto_apibml.md` con rutas, versiones y antipatrones reales.
- Estos archivos se versionan en el repo del proyecto, con `.gitignore` para los que contengan datos sensibles (zona roja).

### Paso 2 — Carga en una sesión diaria

Al iniciar una conversación para trabajar en un ticket de **apibml**, adjuntás estos 3 archivos:

1. `~/my-ai-engine/master_prompt.md` (el motor)
2. `~/proyectos/apibml/preset_laravel.md` (reglas del stack)
3. `~/proyectos/apibml/contexto_apibml.md` (estado actual del proyecto)

Para una sesión genérica sin proyecto específico: solo `master_prompt.md`.

## 3. Instalación

No hay instalación en el sentido tradicional. Los archivos viven en una carpeta de tu elección y los cargas manualmente o con un alias / script.

### 3.1. Estructura del repositorio my-ai-engine

```
~/my-ai-engine/
├─ README.md
├─ master_prompt.md          <- núcleo universal
├─ preset_stack.md           <- plantilla base de stack
├─ contexto_proyecto.template.md <- plantilla base de proyecto
├─ docs/
│   └─ MANUAL_motor_personal.pdf
```

Los presets instanciados (preset\_laravel.md, etc.) y los contextos (contexto\_apibml.md, etc.) viven en el repo de cada proyecto:

```
~/proyectos/apibml/
├─ app/                      <- código Laravel
├─ ...
├─ ai-rules/                 <- carpeta del motor en el proyecto
│   ├── preset_laravel.md    <- instancia de preset_stack.md
│   ├── contexto_apibml.md   <- instancia de contexto_proyecto.template.md
│   └─ docs/                 <- documentación del proyecto (brainstorming, etc.)
```

### 3.2. Pasos

- Descomprimir el ZIP en **~/my-ai-engine/** (o donde prefieras).
- Versionarlo en Git como repo personal. **No** agregar aquí contextos de proyecto con datos sensibles.
- Por cada proyecto nuevo: copiar las plantillas al repo del proyecto, rellenarlas con datos reales y versionarlas ahí.
- Agregar al .gitignore del proyecto los archivos con datos en zona roja/amarilla.
- Opcional: configurar alias en ~/.bashrc para cargar el motor al portapapeles.

### 3.3. Aliases de ejemplo (bash / zsh)

```
# Cargar motor base + preset + contexto de un proyecto al portapapeles
alias motor-apibml='cat ~/my-ai-engine/master_prompt.md \
~/proyectos/apibml/ai-rules/preset_laravel.md \
~/proyectos/apibml/ai-rules/contexto_apibml.md | xclip -selection clipboard'

alias motor-zoho='cat ~/my-ai-engine/master_prompt.md \
~/proyectos/zoho-tracking-system/ai-rules/preset_laravel.md \
~/proyectos/zoho-tracking-system/ai-rules/contexto_zoho_tracking.md | xclip -selection clipboard'
```

*En WSL2 reemplazar xclip -selection clipboard por clip.exe. En macOS, por pbcopy.*

## 4. Uso por plataforma de IA

### 4.1. Claude (web / desktop)

- Crear un **Project** en Claude por cada proyecto técnico (ej. uno para apibml, otro para zoho-tracking).
- Subir master\_prompt.md + el preset + el contexto del proyecto como **knowledge files** del Project. La IA los considera contexto permanente.
- Para sesiones puntuales fuera de un Project: pegar el contenido al inicio del chat dentro de un bloque <contexto>...</contexto>.

### 4.2. ChatGPT

- Crear un **Custom GPT** por proyecto y pegar los 3 archivos en las instrucciones del sistema.
- Alternativa rápida: pegar los archivos en el primer mensaje y pedir confirmación de carga antes de la primera tarea.

### 4.3. Gemini

- Usar **Gems** para cargar el motor de forma persistente.
- Para 1M tokens de contexto: cargar los 3 archivos + el código del proyecto vía Repomix en el primer mensaje.

### 4.4. Modelos locales (Ollama, Cline, OpenCode, etc.)

- Configurar master\_prompt.md como system prompt en la herramienta (Cline, Continue, Open WebUI, OpenCode).
- Cargar preset y contexto al inicio de cada sesión como archivos adjuntos o en el primer mensaje.
- Para cargas grandes preferir modelos con  $\geq 32k$  de contexto (Qwen2.5-Coder 32B, DeepSeek-Coder-V2).

## 5. Workflow por sesión

El flujo recomendado es siempre el mismo, independiente del proyecto:

| Pa<br>so | Acción                                                                 | Resultado esperado                                 |
|----------|------------------------------------------------------------------------|----------------------------------------------------|
| 1        | Cargar master_prompt.md + preset del proyecto + contexto del proyecto. | La IA confirma reglas activas en una línea.        |
| 2        | Describir la tarea con la mínima información necesaria.                | La IA pregunta si falta dato verificable.          |
| 3        | Si la tarea es compleja, escribir <b>modo SoT</b> o <b>/sot</b> .      | La IA responde con bloque SKETCH antes del código. |
| 4        | Aprobar o corregir el SKETCH.                                          | La IA procede a SOLUCIÓN solo con OK explícito.    |
| 5        | Recibir entregable (archivo completo, no fragmentos).                  | Cierre con resumen en $\leq 3$ viñetas.            |

## 6. El trigger SoT

Por defecto la IA responde con su formato directo (estructura primero, implementación después). El protocolo Sketch-of-Thought se activa solo cuando el mensaje contiene uno de estos triggers:

- modo SoT
- /sot
- Frase explícita como «*haz un sketch primero*».

Cuando se activa, la respuesta tiene esta forma:

```
== SKETCH ==  
[META]   : objetivo en una frase  
[LOGIC]  : flujo en pseudocódigo  
[RESTR]  : restricciones / dependencias  
[RIESGO] : punto donde puede romperse  
  
== BREAKPOINT ==  
Si la tarea es grande: la IA se detiene y espera tu OK.  
Si es atómica: continúa sola.  
  
== SOLUCIÓN ==  
[código / output ejecutable]
```

*Cuándo activarlo: refactors multi-archivo, decisiones de arquitectura, migraciones de datos, queries complejas, integraciones externas. Cuándo NO: dudas puntuales, syntax, debugging de un error específico.*

## 7. Reglas de oro (resumen ejecutivo)

Las 12 reglas declaradas en **master\_prompt.md**. Si un asistente las viola, decirle textualmente «violaste la regla N» suele ser suficiente para reencauzarlo.

| #  | Regla                                          |
|----|------------------------------------------------|
| 1  | Nunca supongas; pregunta antes del plan.       |
| 2  | Plan → aprobación → ejecución.                 |
| 3  | Resuelve el error exacto del log primero.      |
| 4  | Respetar lo que ya te di (no repreguntar).     |
| 5  | Archivo completo, no fragmentos.               |
| 6  | Mínima intervención, cero refactor preventivo. |
| 7  | Explica el «por qué», no solo el «cómo».       |
| 8  | Documentación del proyecto = fuente de verdad. |
| 9  | Trazabilidad paso a paso en transformaciones.  |
| 10 | Verifica entorno real antes de prescribir.     |
| 11 | Verifica existencia antes de invocar.          |
| 12 | Vista pre-armada antes que join crudo.         |

## 8. Mantenimiento del motor

### 8.1. Cuándo actualizar

- Cuando detectes una corrección recurrente genérica → agregarla a master\_prompt.md.
- Cuando detectes una corrección recurrente de stack → agregarla al preset del proyecto.
- Cuando un proyecto cambie de stack o entorno → actualizar su contexto en el repo del proyecto.
- Cuando incorpores un stack nuevo → crear preset\_nuevo.md en el repo del proyecto usando preset\_stack.md como base.

### 8.2. Qué NO hacer

- **No** meter reglas de stack específico en master\_prompt.md.
- **No** guardar contextos de proyecto en my-ai-engine/ — pertenecen al repo del proyecto.
- **No** versionar contextos con datos reales de cliente en repos compartidos.
- **No** sobrecargar el master con casos puntuales — si pasa 2 veces en un proyecto, va al preset; si pasa en todos los proyectos, va al master.

### 8.3. Auditoría periódica

Cada ~3 meses, ejecutá esta misma rutina con un asistente: «audité nuestras conversaciones del último trimestre y dame las correcciones recurrentes». Las que se repitan en todos los proyectos suben al master. Las que sean de un stack específico suben al preset. Las obsoletas, bajan.

## 9. Roadmap — presets a crear por proyecto

Estos son ejemplos de presets que podés instanciar a partir de `preset_stack.md` cuando los necesites. Cada uno vive en el repo del proyecto que lo usa, no en `my-ai-engine/`:

| Preset                  | Cuándo crearlo                                                                                   |
|-------------------------|--------------------------------------------------------------------------------------------------|
| <b>preset_oracle.md</b> | Bind variables OCI8, cursores, secuencias, restricciones 10g, gestión de conexiones, savepoints. |
| <b>preset_python.md</b> | Para proyectos FastAPI / Django. Incluir convenciones de tipado, async, manejo de excepciones.   |
| <b>preset_bash.md</b>   | Convenciones para scripts de automatización ( <code>~/ai-workflow/</code> ).                     |
| <b>preset_react.md</b>  | Para proyectos frontend: convenciones de componentes, hooks, manejo de estado.                   |

*Para crear un contexto de proyecto nuevo: copiar `contexto_proyecto.template.md` al repo del proyecto y rellenar todas las secciones antes de la primera sesión de trabajo.*

## 10. Filosofía del motor

Este sistema no es para que la IA sea más lista. Es para que **vos** dejes de repetir lo mismo en cada chat. La IA solo refleja lo que le diste; si no le diste reglas, te devuelve genéricos.

El valor del motor crece con el uso: cada corrección que codifiques en un archivo es una corrección que no tendrás que hacer nunca más. Tratalo como código: versionalo, refactorízalo, mantenelo limpio.

*Documento generado a partir de la auditoría cruzada de Gemini, ChatGPT, DeepSeek y Claude — mayo 2026. v2.0: estructura simplificada a 3 pilares, contextos de proyecto migrados a sus respectivos repositorios.*